

http://algs4.cs.princeton.edu

3.4 HASH TABLES

- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

Symbol table implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	`compareTo()`

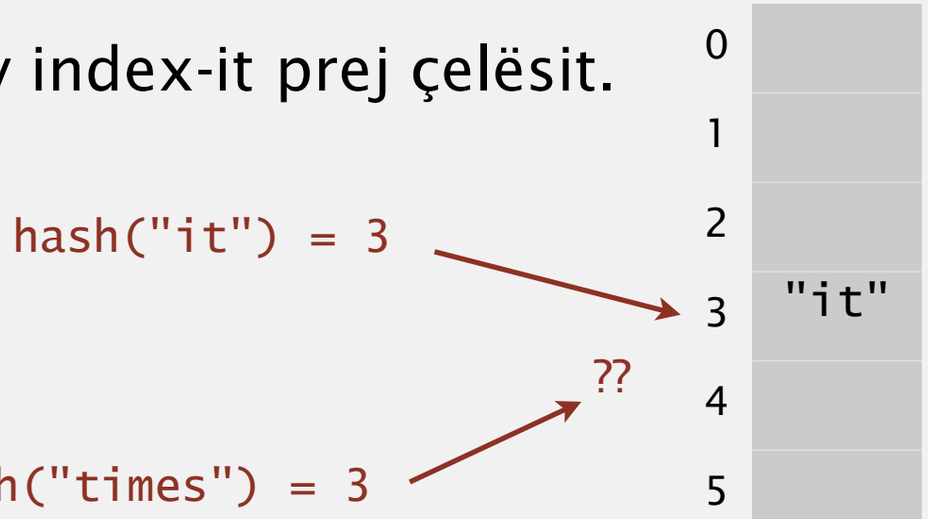
Q. A mund të ketë kompleksitet më të mire sesa logaritmik?

A. Po, mirëpo me një lloj tjetër të qasjes në të dhëna.

Hashing: plani bazë

Të ruhen elementetnë një key-indexed tabelë (ku index është një funkcion i çelësit (key)).

Hash funkcioni. Metodë për llogaritjen e array index-it prej çelësit.



Mangësitë.

- Llogaritja e hash funkcionit.
- Equality testi: Metoda për të [rovuar nëse dy çelësa janë equal (të barabartë).
- Zgjidhja e kolizioneve: Përdoren algoritmet dhe strukturat e të dhënave, për t'u trajtuar dy çelësat që hashohen në të njejtin index të array (figura lartë).



3.4 Hash Tables

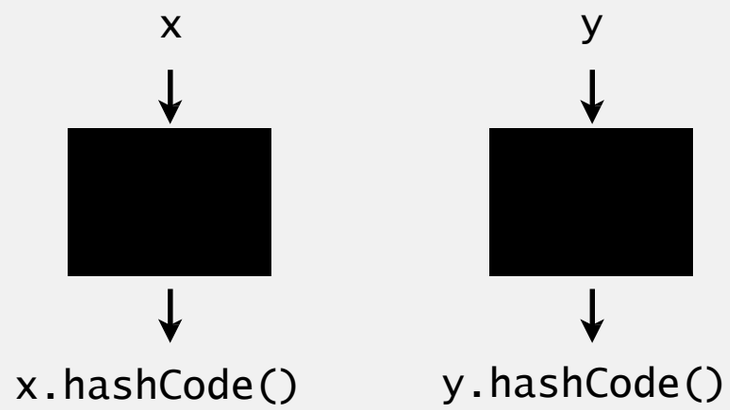
- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

Hash code në Java

Të gjitha Java klasat trashëgojnë një metodë `hashCode()`, e cila kthen një int 32-bit.

Kërkesa. Nëse `x.equals(y)`, atëherë `(x.hashCode() == y.hashCode())`.

Preferohet. Nëse `!x.equals(y)`, atëherë `(x.hashCode() != y.hashCode())`.



Implementimi i hash code: integer, boolean dhe double

Java library implementimet

```
public final class Integer
{
   private final int value;
   ...

   public int hashCode()
   {  return value;  }
}
```

```
public final class Boolean
{
   private final boolean value;
   ...

   public int hashCode()
   {
      if (value) return 1231;
      else       return 1237;
   }
}
```

```
public final class Double
{
   private final double value;
   ...

   public int hashCode()
   {
      long bits =
      doubleToLongBits(value); return
      (int) (bits ^ (bits >>> 32));
   }
}
```

convert to IEEE 64-bit representation;
xor most significant 32-bits
with least significant 32-bits

Warning: -0.0 and +0.0 have different hash codes

Implementimi i hash code: strings

Java library implementimi

```
public final class String
{
   private final char[] s;
   ...

   public int hashCode()
   {
      int hash = 0;
      for (int i = 0; i < length(); i++)
         hash = s[i] + (31 * hash);
      return hash;
   }
}
```

ith character of s

char	Unicode
…	…
'a'	97
'b'	98
'c'	99
…	…

- Metoda e Horner-it për të hash string me length L: L multiplies/adds.
- Equivalent to $h = s[0] \cdot 31^{L-1} + \ldots + s[L-3] \cdot 31^2 + s[L-2] \cdot 31^1 + s[L-1] \cdot 31^0$.

P.sh.

```
String s = "call";
int code = s.hashCode();
```

3045982 = 99·313 + 97·312 + 108·311 + 108·310
= 108 + 31· (108 + 31 · (97 + 31 · (99)))
(Horner's method)

Implementimi i hash code: strings

Optimizimi i performancës.

- Cache hash vlerën në një variable të instancës.
- Return cached vlerën.

```
public final class String
{
   private int hash = 0;
   private final char[] s;
   ...

   public int hashCode()
   {
      int h = hash;
      if (h != 0) return h;
      for (int i = 0; i < length(); i++)
         h = s[i] + (31 * h);
      hash = h;
      return h;
   }
}
```

cache of hash code

return cached value

store cache of hash code

Implementimi i hash code: tipet e definuara nga përdoruesi

```
public final class Transaction implements Comparable<Transaction>
{
   private final String  who;
   private final Date    when;
   private final double  amount;

   public Transaction(String who, Date when, double  amount)
   {  /* as before */  }

   ...

   public boolean equals(Object y)
   {  /* as before */  }

   public int hashCode()
   {
      int hash = 17;
      hash = 31*hash + who.hashCode();
      hash = 31*hash + when.hashCode();
      hash = 31*hash + ((Double) amount).hashCode();
      return hash;
   }
}
```

nonzero constant

for reference types, use hashCode()

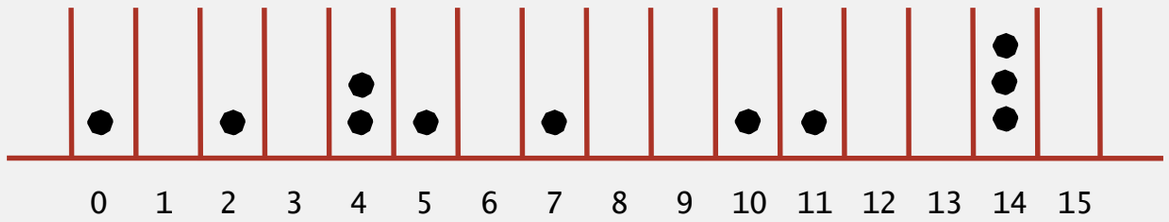
for primitive types, use hashCode() of wrapper type

typically a small prime

Uniform hashing - supozimi

Uniform hashing supozimi. Çdo çelës është me gjasë të njejtë që të hash në një integer në mes të 0 dhe $M - 1$.

Topat dhe koshat. Gjuhen topat uniformisht në mënyrë të rastësishme në M kosha.



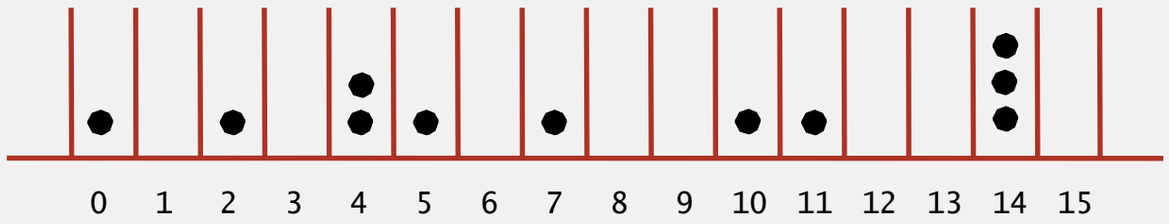
Birthday problem. Pritet që dy topa përfundojnë në të njejtin kosh pas ~ $\sqrt{\pi M / 2}$ tosses.
70 = 99.9%

Pritet që çdo kosh ka ≥ 1 top pas ~ $M \ln M$ gjuajtjeve.

Uniform hashing - supozimi

Uniform hashing supozimi. Çdo çelës është me gjasë të njejtë që të hash në një integer në mes të 0 dhe $M - 1$.

Topat dhe koshat. Gjuhen topat uniformisht në mënyrë të rastësishme në M kosha





Java's `String` data uniformly distribute the keys of Tale of Two Cities



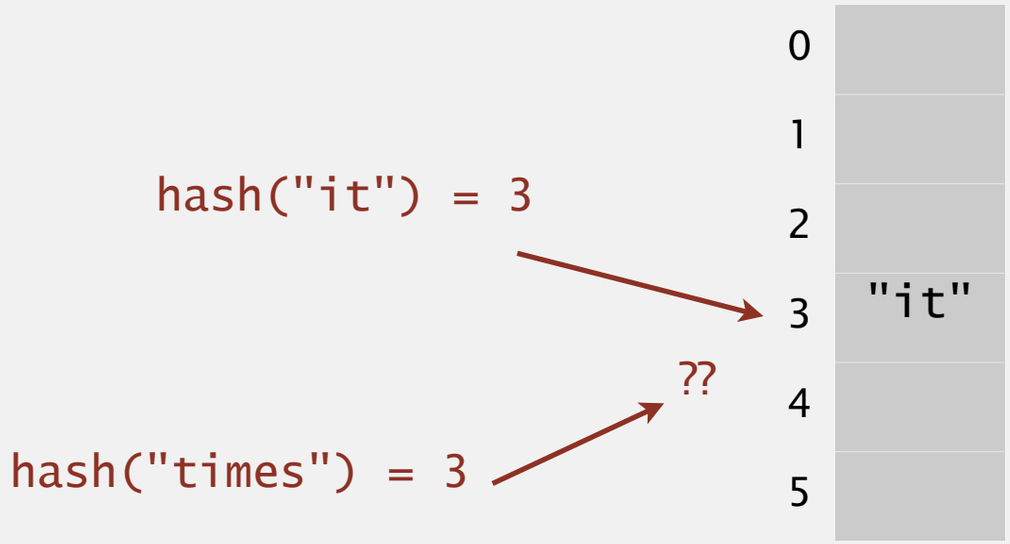
3.4 Hash Tables

- *hash functions*
- *separate chaining*
- *linear probing*
- *context*

Kolizionet

Kolizioni. Dy çelësa të veçantë hashohen në indeks të njejtë.

- Bazuar në Birthday problem Nuk mund të anashkalohen kolizionet përveç Nëse shpenzohet sasi qesharake e memories (kuadratike).



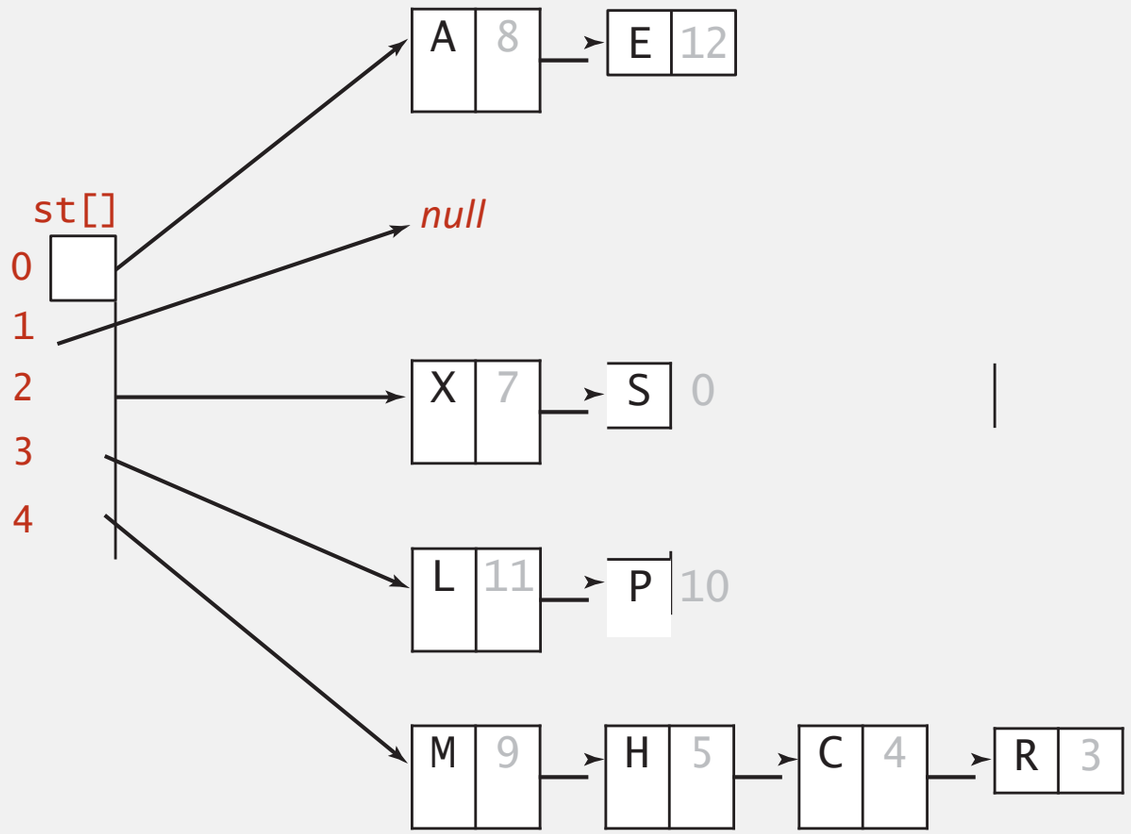
Sfida. Të trajtohen kolizionet në mënyrë efikase

Separate-chaining symbol table

Përdoret një array me $M < N$ linked lista. [H. P. Luhn, IBM 1953]

- Hash: mapohet çelësi integer i në mes të 0 dhe $M - 1$.
- Insert: put në fillim të zingjirit të $i^{të}$ ose në fund (preferohet në fillim).
- Search: Duhet të search vetëm zingjiri i $i^{të}$.

key	hash	value
S	2	0
E	0	1
A	0	2
R	4	3
C	4	4
H	4	5
E	0	6
X	2	7
A	0	8
M	4	9
P	3	10
L	3	11
E	0	12



Separate-chaining symbol table: Java implementimi

```
public class SeparateChainingHashST<Key, Value>
{
   private int M = 97;                  // number of chains
   private Node[] st = new Node[M];      // array of chains

   private static class Node
   {
      private Object key;
      private Object val;
      private Node next;
      ...
   }

   private int hash(Key key)
   {  return (key.hashCode() & 0x7fffffff) % M;  }

   public Value get(Key key) {
      int i = hash(key);
      for (Node x = st[i]; x != null; x = x.next)
         if (key.equals(x.key)) return (Value) x.val;
      return null;
   }

}
```

array doubling and halving code omitted

no generic array creation (declare key and value of type Object)

Separate-chaining symbol table: Java implementimi

```
public class SeparateChainingHashST<Key, Value>
{
   private int M = 97;                // number of chains
   private Node[] st = new Node[M];  // array of chains

   private static class Node
   {
      private Object key;
      private Object val;
      private Node next;
      ...
   }

   private int hash(Key key)
   {  return (key.hashCode() & 0x7fffffff) % M;   }

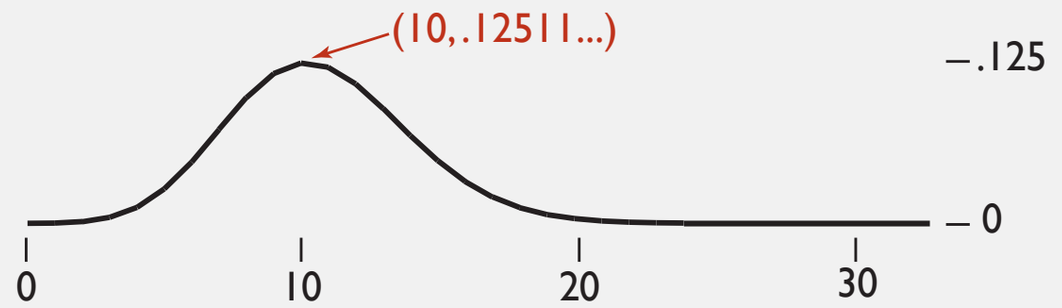
   public void put(Key key, Value val) {
      int i = hash(key);
      for (Node x = st[i]; x != null; x = x.next)
         if (key.equals(x.key)) { x.val = val; return; }
      st[i] = new Node(key, val, st[i]);
   }

}
```

Analiza e separate chaining

Proposition. Nën supozimin për shpërndarje uniform tek hashing, probabiliteti që numri i çelësave është në faktorë konstant prej N / M është jashtëzakonisht afër 1shit.

Distribuimi i list size bëhet sipas distribuimit binomial.



Binomial distribution ($N = 10^4$, $M = 10^3$, $a = 10$)

equals() and hashCode()

Rezultati. Numri i tentimeve për search/insert ësthë proporcional me N / M.

- M i madh Shumë zingjirë të zbrazët.
- M i vogël Zingjirët shumë të gjatë.
- Zgjedhja tipike: $M \sim N / 4 \Rightarrow$ operacione në kohë konstante.

M times faster than sequential search

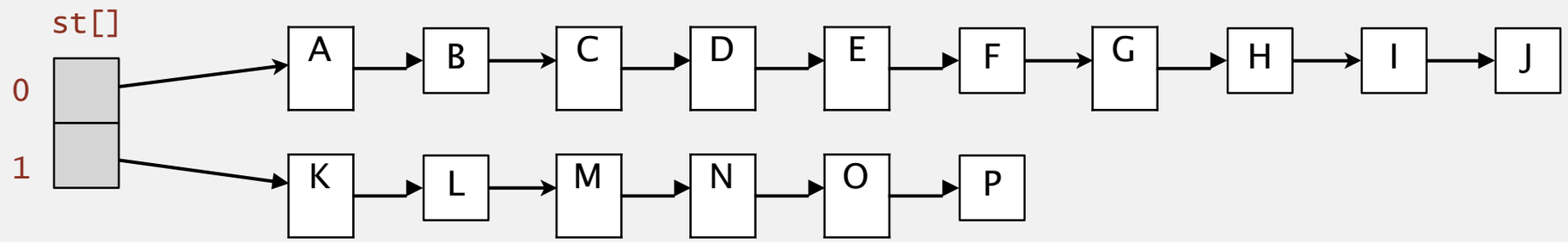
Resizing in a separate-chaining hash table

Qëllimi. Length mesatar i listës N / M = konstant.

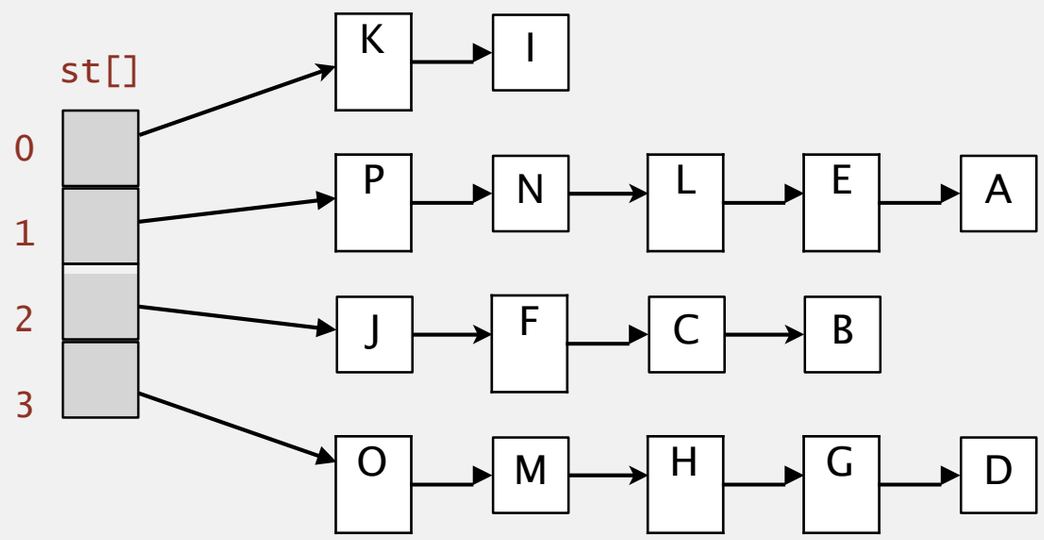
- Dyfishohet size i array M kur $N / M \geq 8$.
- Përgjysmohet size i array M kur $N / M \leq 2$.
- Kur ka resizing duhet të rehash të gjithë çelësat.

← `x.hashCode()` does not change but `hash(x)` can change

before resizing



after resizing

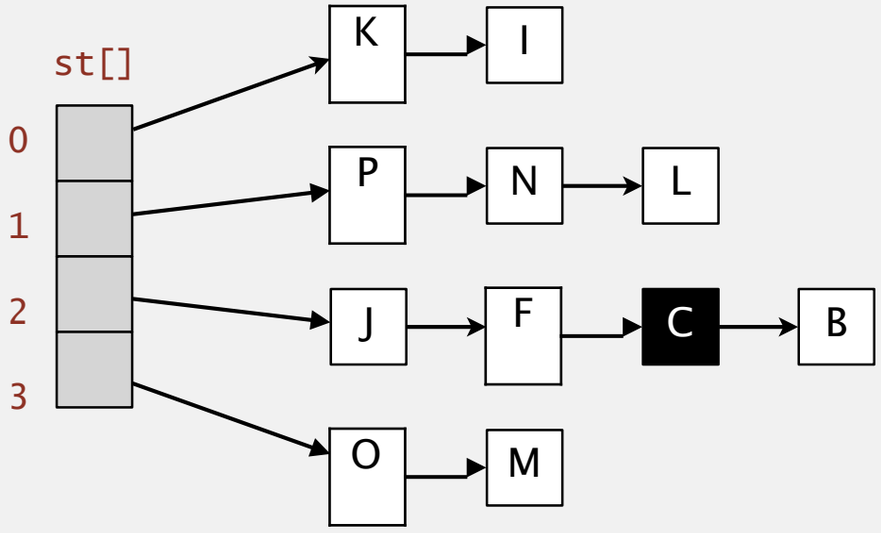


Deletion në një separate-chaining hash table

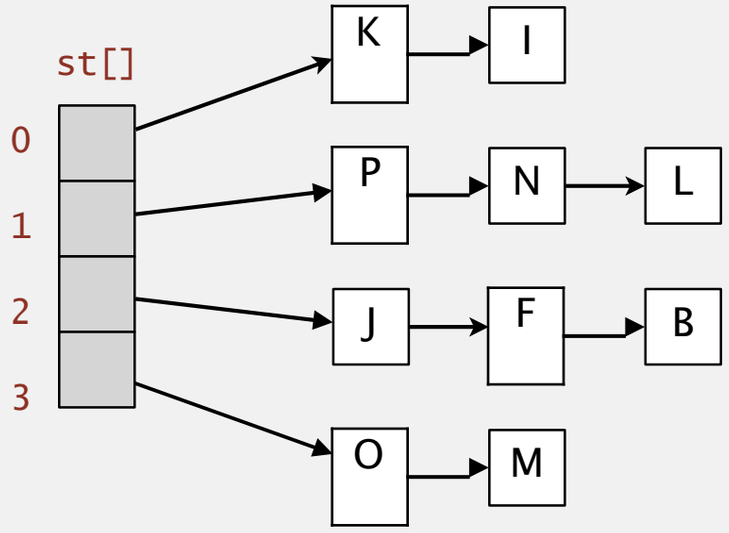
Q. Si tëdelete një çelës (dhe vlera përcjellëse)?

A. Vetëm largohet zingjiri që lidh çelësin në fjalë.

before deleting C



after deleting C



Symbol table implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	`compareTo()`
separate chaining	N	N	N	3-5 *	3-5 *	3-5 *		`equals()` `hashCode()`

* under uniform hashing assumption



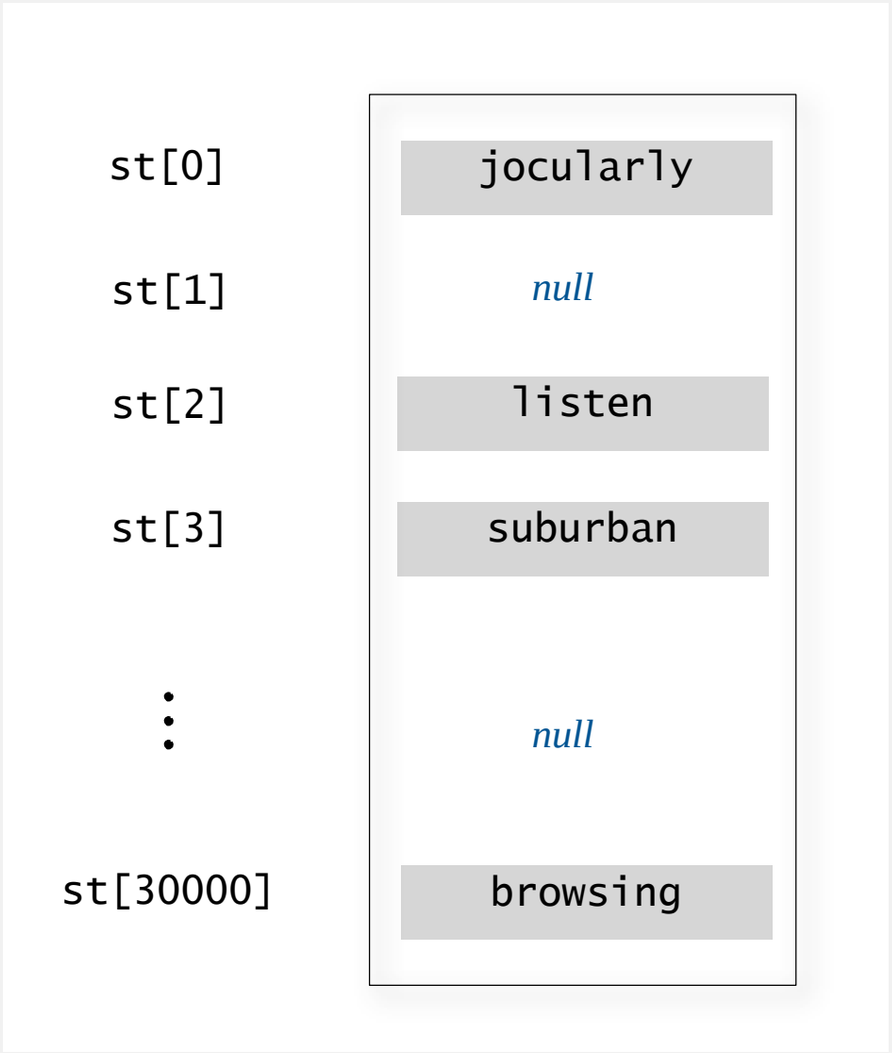
3.4 Hash Tables

- hash functions
- separate chaining
- linear probing
- context

Zgjidhja e kolizioneve: open addressing

Open addressing. [Amdahl-Boehme-Rocherster-Samuel, IBM 1953]

Kur çelësi i ri hasë në kolizion, gjindet një vend i zbrazët, dhe vendoset aty



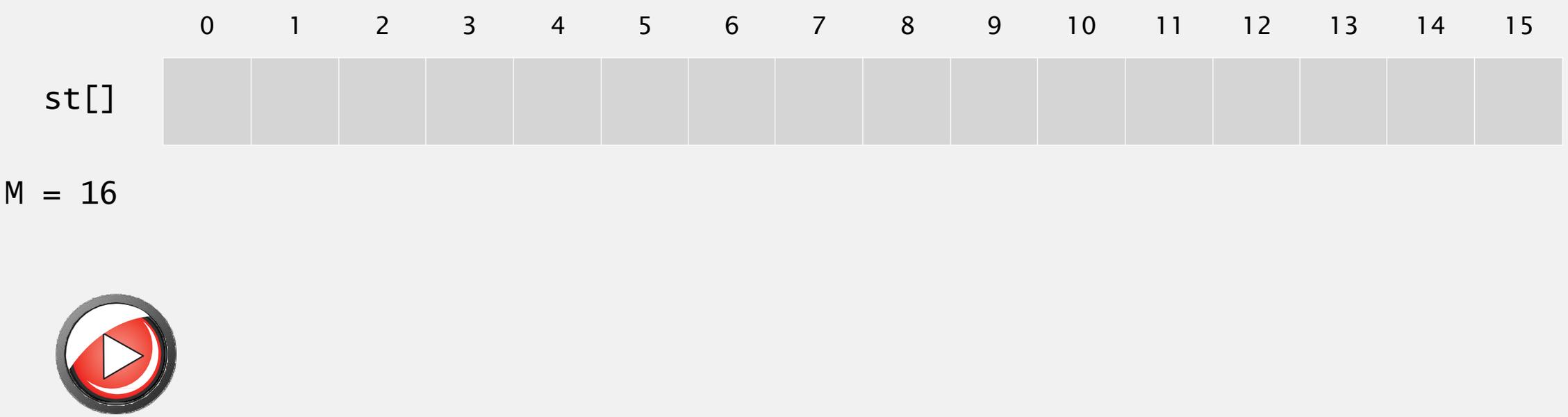
linear probing (M = 30001, N = 15000)

Linear-probing hash table demo

Hash. Mapohet çelësi në integer `i` në mes të 0 dhe `M-1`.

Insert. Put në table index `i` nëse ka vend; nëse nuk ka, try (tenton) `i+1`, `i+2`, etj.

linear- probing hash table



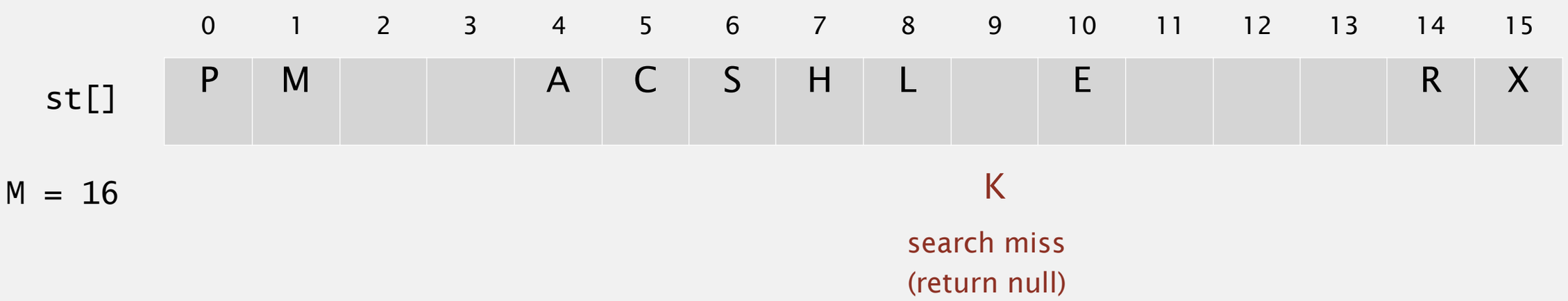
Linear-probing hash table demo

Hash. Mapohet çelësi në integer `i` në mes të 0 dhe `M-1`.

Search. Put në table index `i` nëse ka vend; nëse nuk ka, try (tenton) `i+1`, `i+2`, etj.

search K

hash(K) = 5

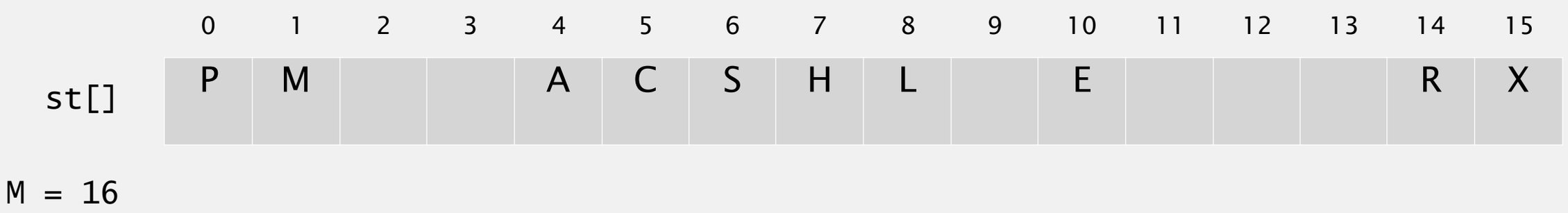


Linear-probing hash table - përmbledhje

Hash. Mapohet çelësi në integer `i` në mes të 0 dhe `M-1`.
Insert. Put në table index `i` nëse ka vend; nëse nuk ka, try (tenton) `i+1`, `i+2`, etj.

Search. Search table index `i`; nëse është zënë, por pa match, try `i+1`, `i+2`, etj.

Note. Array size `M` duhet të jetë më e madhe sesa numri i çelës-vlerë çifteve `N`.



Linear-probing symbol table: Java implementimi

```
public class LinearProbingHashST<Key, Value>
{
   private int M = 30001;
   private Value[] vals = (Value[]) new Object[M];
   private Key[]   keys = (Key[])   new Object[M];

   private int hash(Key key)              {  /* as before */   }

   private void put(Key key, Value val) {  /* next slide */  }

   public Value get(Key key)
   {
      for (int i = hash(key); keys[i] != null; i =  (i+1) % M)
         if (key.equals(keys[i]))
             return vals[i];
      return null;
   }



}
```

array doubling and halving code omitted

Linear-probing symbol table: Java implementimi

```
public class LinearProbingHashST<Key, Value>
{
   private int M = 30001;
   private Value[] vals = (Value[]) new Object[M];
   private Key[]   keys = (Key[])   new Object[M];

   private int hash(Key key)   { /* as before      */  }

   private Value get(Key key)  { /* previous slide */  }

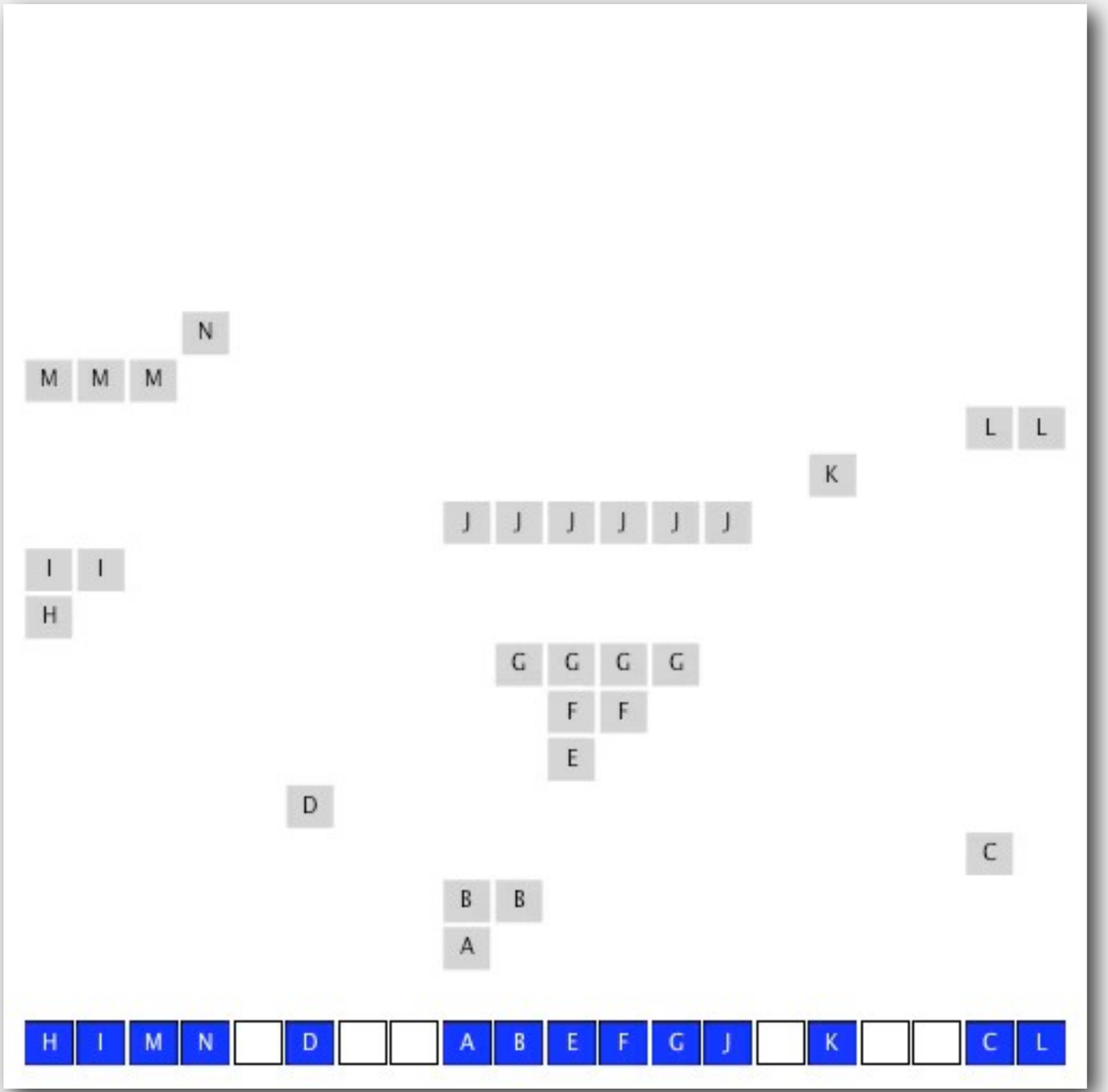
   public void put(Key key, Value val)
   {
      int i;
      for (i = hash(key); keys[i] != null; i = (i+1) % M)
         if (keys[i].equals(key))
             break;
      keys[i] = key;
      vals[i] = val;
   }

}
```

Clustering (grumbullimi)

Cluster. Një bllok i elementeve fqinje.

Observimi. Çelësat e ri kanë tendencë që të hashohen në mes të cluster-ave të mëdhenj (teoria e prob.).



Resizing në një linear-probing hash table

Qëllimi. Length mesatar i listës $N / M \leq ½$.

- Dyfishohet size i array M kur $N / M \geq ½$.
- Përgjysmohet size i array M kur $N / M \leq ⅛$.
- Kur ka resizing duhet të rehash të gjithë çelësat.

before resizing



after resizing



ST implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	`compareTo()`
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	`compareTo()`
separate chaining	N	N	N	3-5 *	3-5 *	3-5 *		`equals()` `hashCode()`
linear probing	N	N	N	3-5 *	3-5 *	3-5 *		`equals()` `hashCode()`

* under uniform hashing assumption



3.4 Hash Tables

- *hash functions*
- *separate chaining*
- *linear probing*

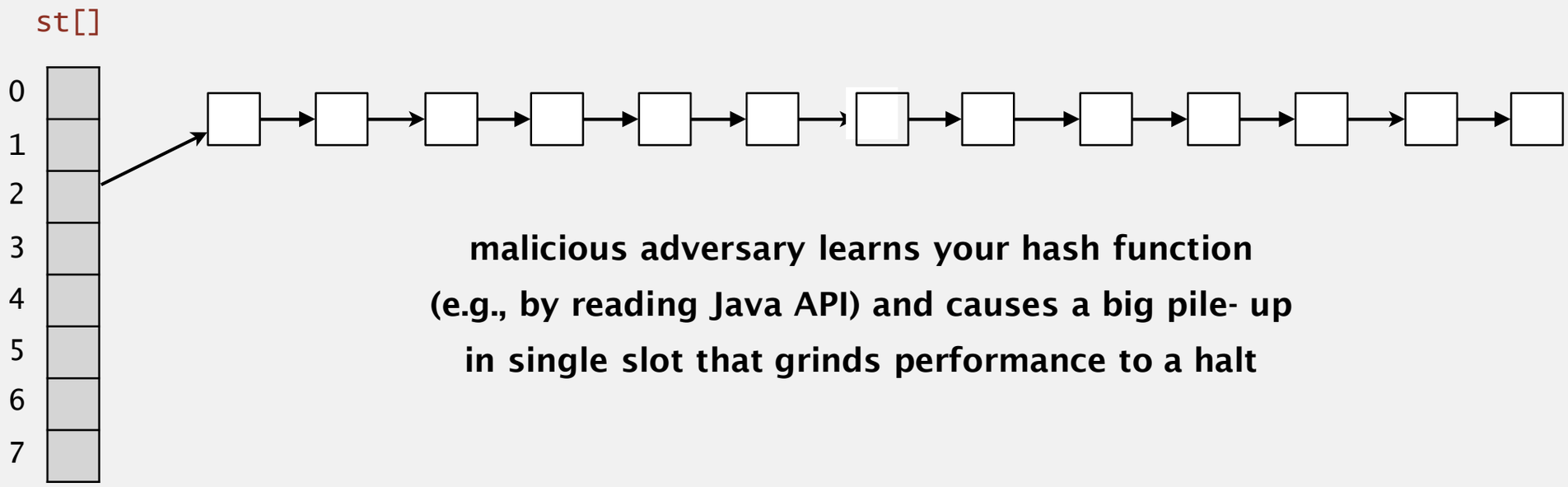
context

War story: algorithmic complexity attacks

Q. Is the uniform hashing assumption important in practice?

A. Obvious situations: aircraft control, nuclear reactor, pacemaker.

A. Surprising situations: denial-of-service attacks.



Real-world exploits. [Crosby-Wallach 2003]

- Bro server: send carefully chosen packets to DOS the server, using less bandwidth than a dial-up modem.
- Perl 5.8.0: insert carefully chosen strings into associative array.
- Linux 2.4.20 kernel: save files with carefully chosen names.

War story: algorithmic complexity attacks

A Java bug report.

Jan Lieskovsky 2011-11-01 10:13:47 EDT **Description**

Julian Wälde and Alexander Klink reported that the String.hashCode() hash function is not sufficiently collision resistant. hashCode() value is used in the implementations of HashMap and Hashtable classes:

http://docs.oracle.com/javase/6/docs/api/java/util/HashMap.html
http://docs.oracle.com/javase/6/docs/api/java/util/Hashtable.html

A specially-crafted set of keys could trigger hash function collisions, which can degrade performance of HashMap or Hashtable by changing hash table operations complexity from an expected/average O(1) to the worst case O(n). Reporters were able to find colliding strings efficiently using equivalent substrings and meet in the middle techniques.

This problem can be used to start a denial of service attack against Java applications that use untrusted inputs as HashMap or Hashtable keys. An example of such application is web application server (such as tomcat, see bug #750521) that may fill hash tables with data from HTTP request (such as GET or POST parameters). A remote attack could use that to make JVM use excessive amount of CPU time by sending a POST request with large amount of parameters which hash to the same value.

This problem is similar to the issue that was previously reported for and fixed
in e.g. perl:
http://www.cs.rice.edu/~scrosby/hash/CrosbyWallach_UsenixSec2003.pdf

Algorithmic complexity attack on Java

Goal. Find family of strings with the same hash code.

Solution. The base-31 hash code is part of Java's string API.

key	hashCode()
"Aa"	2112
"BB"	2112

key	hashCode()
"AaAaAaAa"	-540425984
"AaAaAaBB"	-540425984
"AaAaBBAa"	-540425984
"AaAaBBBB"	-540425984
"AaBBAaAa"	-540425984
"AaBBAaBB"	-540425984
"AaBBBBAa"	-540425984
"AaBBBBBB"	-540425984

key	hashCode()
"BBAaAaAa"	-540425984
"BBAaAaBB"	-540425984
"BBAaBBAa"	-540425984
"BBAaBBBB"	-540425984
"BBBBAaAa"	-540425984
"BBBBAaBB"	-540425984
"BBBBBBAa"	-540425984
"BBBBBBBB"	-540425984

2^N strings of length 2N that hash to same value!

Diversion: one-way hash functions

One-way hash function. "Hard" to find a key that will hash to a desired value (or two keys that hash to same value).

Ex. MD4, MD5, SHA-0, SHA-1, SHA-2, WHIRLPOOL, RIPEMD-160, ...

known to be insecure

```
String password = args[0];
MessageDigest sha1 = MessageDigest.getInstance("SHA1");
byte[] bytes = sha1.digest(password);

/* prints bytes as hex string */
```

Applications. Digital fingerprint, message digest, storing passwords.

Caveat. Too expensive for use in ST implementations.

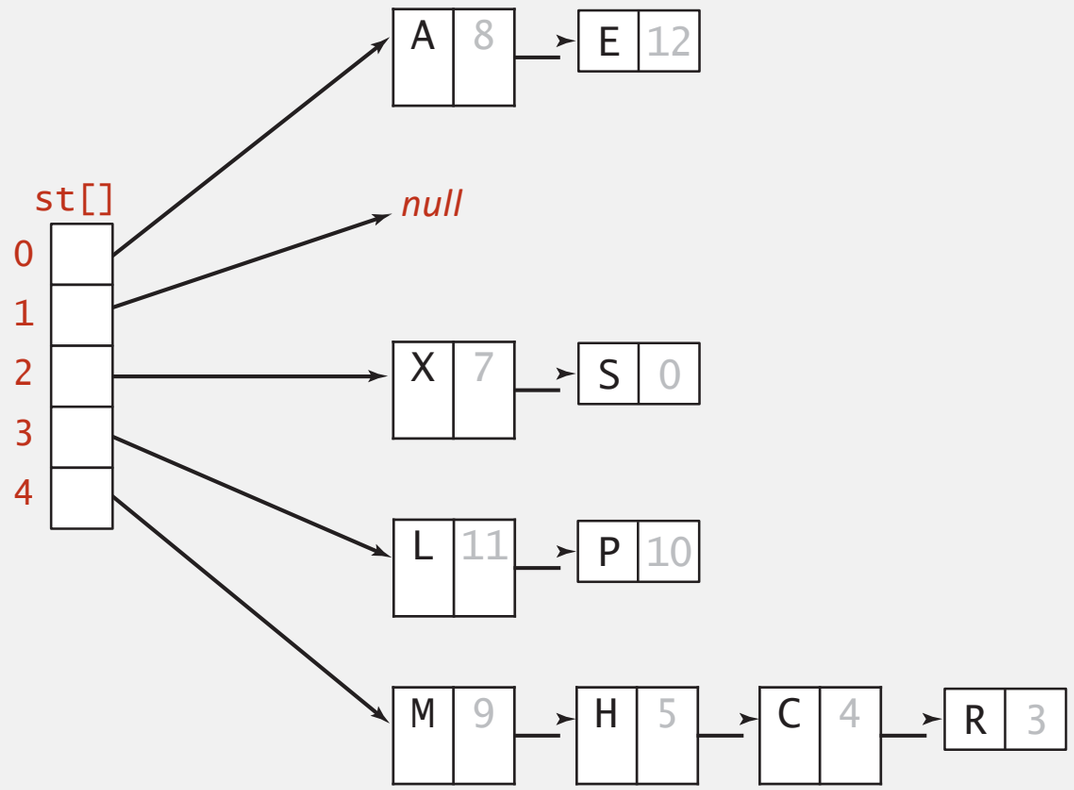
Separate chaining vs. linear probing

Separate chaining.

- Performance degrades gracefully.
- Clustering less sensitive to poorly-designed hash function.

Linear probing.

- Less wasted space.
- Better cache performance.



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C	S	H	L		E				R	X
vals[]	10	9			8	4	0	5	11		12				3	7

Hashing: variations on the theme

Many improved versions have been studied.

Two-probe hashing. [separate-chaining variant]

- Hash to two positions, insert key in shorter of the two chains.
- Reduces expected length of the longest chain to $\log \log N$.

Double hashing. [linear-probing variant]

- Use linear probing, but skip a variable amount, not just 1 each time.
- Effectively eliminates clustering.
- Can allow table to become nearly full.
- More difficult to implement delete.

Cuckoo hashing. [linear-probing variant]

- Hash key to two positions; insert key into either position; if occupied, reinsert displaced key into its alternative position (and recur).
- Constant worst-case time for search.



Hash tables vs. balanced search trees

Hash tables.

- Simpler to code.
- No effective alternative for unordered keys.
- Faster for simple keys (a few arithmetic ops versus $\log N$ compares).
- Better system support in Java for strings (e.g., cached hash code).

Balanced search trees.

- Stronger performance guarantee.
- Support for ordered ST operations.
- Easier to implement `compareTo()` correctly than `equals()` and `hashCode()`.

Java system includes both.

- Red-black BSTs: `java.util.TreeMap`, `java.util.TreeSet`.
- Hash tables: `java.util.HashMap`, `java.util.IdentityHashMap`.